



UNIVERSITÉ CADI AYYAD
Faculté des Sciences Semlalia de Marrakech

RAPPORT DE MINI-PROJET EN C++

Filière : Informatique

Présenté par :

NAIM Oumayma

RAOUZI Oumaima

Sujet

Application de jeu de cache en C++

Encadré par :

Pr. Rachida HANNANE

Remerciements

Avant de présenter ce mini-projet, nous tenons à exprimer nos sincères remerciements à notre professeure de programmation C++, **Mme Rachida HANNANE**, pour la qualité de son enseignement, sa disponibilité et ses précieux conseils.

Nous lui adressons notre profonde gratitude pour son accompagnement, ses encouragements et les connaissances qu'elle nous a transmises tout au long de ce travail. Grâce à son soutien et à ses explications claires, nous avons pu mieux comprendre plusieurs notions importantes en programmation C++ et mener à bien la réalisation de ce mini-projet.

Nous la remercions vivement pour son sérieux, sa patience et son engagement pédagogique, qui ont été d'une grande aide dans notre apprentissage.

NAIM et RAOUZI

Résumé

Ce mini-projet présente la conception et la réalisation d'une application de jeu de cache en 2D développée en langage C++ à l'aide de la bibliothèque **SFML** (*Simple and Fast Multimedia Library*). L'objectif principal de ce travail est de mettre en pratique les notions fondamentales de la programmation orientée objet, ainsi que les techniques de base nécessaires à la création d'une application graphique et interactive.

La réalisation de ce projet nous a permis d'exploiter plusieurs concepts importants, notamment la gestion des fenêtres graphiques, l'affichage des éléments visuels, la gestion des événements du clavier, les déplacements du personnage, la détection des collisions, ainsi que l'interaction entre les différents composants du jeu. Nous avons également appris à mieux structurer une application en plusieurs classes afin de rendre le code plus clair, organisé et facile à maintenir.

Ce mini-projet représente une expérience enrichissante, car il nous a permis de consolider nos compétences en C++, de découvrir l'utilisation pratique de la bibliothèque SFML dans le développement d'applications 2D, et de mieux comprendre les différentes étapes de conception et d'implémentation d'un jeu simple. Il nous a aussi aidées à développer notre esprit d'analyse, notre logique de programmation et notre capacité à résoudre les problèmes rencontrés au cours du développement.

Mots-clés : C++, SFML, programmation orientée objet, jeu de cache, 2D, application interactive, développement logiciel.

Liste des abréviations

C++ Langage de programmation C++

POO Programmation orientée objet

SFML Simple and Fast Multimedia Library

2D Deux dimensions

UML Unified Modeling Language (Langage de modélisation unifié)

StarUML Outil de modélisation UML

Table des figures

3.1	Langage de programmation C++	12
3.2	Bibliothèque SFML	12
3.3	Environnement de développement Visual Studio	13
3.4	Environnement de modélisation StarUML	13
4.1	Diagramme de classes du jeu de cachette en 2D	15
4.2	Menu principal du jeu	17
4.3	Page « À propos » du jeu	17
4.4	Choix du personnage	18
4.5	Interface du jeu en cours	18
4.6	Écran de pause du jeu	19
4.7	Écran de fin du jeu (défaite)	19
4.8	Écran de victoire	20

Table des matières

Remerciements	I
Résumé	II
Introduction générale	1
1 Contexte général du projet	3
1.1 Introduction	3
1.2 Problématique	4
1.3 Objectif Principal du projet	5
1.4 Planification du projet	5
1.5 Conclusion	6
2 Analyse du projet	7
2.1 Introduction	7
2.2 Description du jeu	7
2.3 Fonctionnement du jeu	8
2.4 Règles du jeu	9
2.5 Conclusion	9
3 Outils et technologies utilisés	11
3.1 Introduction	11
3.2 Le langage C++	11
3.3 La bibliothèque SFML	12
3.4 Environnement de développement : Visual Studio	12
3.5 Environnement de modélisation : StarUML	13
3.6 Conclusion	13

4	Conception et réalisation	14
4.1	Introduction	14
4.2	Architecture du projet	14
4.3	Diagramme de classes	15
4.4	Description du diagramme de classes	15
4.5	Fonctionnement du jeu	16
4.6	Présentation des interfaces du jeu	16
4.7	Conclusion	20
	Conclusion et perspectives	21

Introduction générale

Dans le cadre de notre formation en informatique à la Faculté des Sciences et Techniques de Marrakech, nous avons réalisé un mini-projet en programmation C++. Ce projet a pour objectif de mettre en pratique les connaissances acquises durant le module de programmation, notamment en programmation orientée objet.

Le projet consiste en la conception et le développement d'un jeu de cachette en deux dimensions (2D) en utilisant le langage C++ et la bibliothèque **SFML** (Simple and Fast Multimedia Library). Ce type de projet permet de mieux comprendre les concepts fondamentaux liés au développement d'applications interactives, tels que la gestion des événements, l'affichage graphique, le déplacement des objets, ainsi que la détection des collisions.

La réalisation de ce jeu nous a permis de renforcer nos compétences en programmation, d'améliorer notre logique algorithmique et de mieux comprendre la structuration d'un programme en plusieurs classes. Elle nous a également permis de découvrir l'utilisation d'une bibliothèque graphique comme SFML pour créer une interface visuelle simple et dynamique.

Organisation du rapport

Ce rapport, qui présente le travail réalisé lors de ce mini-projet, est organisé comme suit :

- Le premier chapitre présente le contexte général du projet ainsi que les objectifs visés.
- Le deuxième chapitre est consacré à l'analyse des besoins et à la description du fonctionnement du jeu.
- Le troisième chapitre décrit les outils et technologies utilisés, notamment le langage C++ et la bibliothèque SFML.

Introduction générale

- Le quatrième chapitre présente la conception et la réalisation du jeu, ainsi que les principales fonctionnalités développées.
- Enfin, nous terminons ce travail par une conclusion générale résumant les résultats obtenus et les perspectives d'amélioration.

Chapitre 1

Contexte général du projet

Dans ce chapitre, nous présentons le cadre général de notre projet. Nous allons décrire le contexte dans lequel s'inscrit ce travail, la problématique rencontrée, ainsi que les objectifs visés par la réalisation de ce mini-projet.

1.1. Introduction

Les jeux en 2D représentent un excellent moyen de mettre en pratique les bases de la programmation, car ils permettent de travailler sur plusieurs notions importantes, telles que la gestion des événements, l'affichage graphique, les déplacements ainsi que les collisions. Dans ce cadre, le projet, intitulé **Jeu de cachette en 2D**, a été conçu dans le but d'appliquer les connaissances acquises lors du module de programmation C++. Sa réalisation consiste à développer une application graphique interactive en utilisant le langage C++ et la bibliothèque **SFML**.

L'objectif principal est de mettre en place une application simple, organisée, esthétique et attrayante, permettant à l'utilisateur de jouer dans un environnement dynamique. Le jeu assure la gestion du déplacement du personnage, la présence des obstacles, la détection des collisions ainsi que l'interaction avec les différents éléments graphiques.

Ce travail nous a permis de mieux comprendre la programmation orientée objet et le fonctionnement d'une application interactive. Il nous a également aidées à renforcer nos compétences en programmation, à mieux organiser notre code et à découvrir les différentes étapes nécessaires à la conception et à la réalisation d'un jeu simple en 2D.

1.2. Problématique

De nos jours, il arrive souvent que les utilisateurs cherchent un moyen simple et amusant pour se divertir et passer le temps. En situation d'ennui, les jeux interactifs représentent une solution intéressante, car ils permettent à l'utilisateur de se détendre tout en profitant d'une expérience dynamique et accessible.

Dans ce contexte, nous avons choisi de développer un jeu de cachette en 2D capable d'offrir un environnement ludique, simple et attrayant. Cependant, la réalisation de ce type de jeu ne se limite pas seulement à l'aspect divertissant, car elle soulève également plusieurs difficultés techniques liées à la gestion des différents éléments du jeu et à leur interaction.

Tout d'abord, la gestion des déplacements du personnage a constitué un défi important, notamment pour assurer un mouvement fluide et cohérent. Il a également été nécessaire de gérer correctement les événements du clavier afin de permettre au joueur d'interagir efficacement avec le jeu.

Ensuite, la détection des collisions entre le personnage et les obstacles a posé certaines difficultés. En effet, il était essentiel de garantir que les collisions soient détectées avec précision afin d'éviter des comportements incorrects, comme le passage à travers les obstacles ou des pertes non justifiées.

Par ailleurs, l'affichage graphique à l'aide de la bibliothèque **SFML** a nécessité une bonne organisation du code et une gestion rigoureuse des ressources, telles que les images et les animations. Une mauvaise gestion pouvait entraîner des problèmes d'affichage ou de performance.

Enfin, l'organisation globale du programme en plusieurs classes a demandé une réflexion approfondie afin de structurer le code de manière claire, lisible et facile à maintenir. De la même manière, une bonne structuration des données est essentielle afin d'éviter les redondances et les incohérences. Dans ce cadre, les principes de normalisation jouent un rôle important, puisqu'ils permettent de décomposer correctement les relations en fonction des dépendances fonctionnelles, tout en assurant une décomposition sans perte d'information et sans perte de dépendances fonctionnelles.

Ainsi, la problématique principale de ce projet consiste à concevoir et développer un jeu de cachette en 2D à la fois divertissant et fonctionnel, capable de permettre à l'utilisateur de passer le temps agréablement, tout en assurant une bonne fluidité, une gestion correcte des

interactions et une structure de code bien organisée.

1.3. Objectif Principal du projet

L'objectif principal de ce mini-projet est de concevoir et de développer un jeu de cache en 2D en utilisant le langage **C++** et la bibliothèque **SFML**.

À travers ce projet, nous cherchons à réaliser une application interactive capable de gérer le déplacement du personnage, les obstacles et la détection des collisions, tout en garantissant un affichage graphique fluide et cohérent.

Ce travail nous permet également de mettre en pratique les concepts de la programmation orientée objet, d'améliorer la structuration du code et de renforcer nos compétences dans le développement d'applications graphiques.

1.4. Planification du projet

La réalisation de ce mini-projet s'est déroulée en plusieurs étapes. Dans un premier temps, nous avons défini l'idée générale du jeu ainsi que ses principales fonctionnalités. Cette étape nous a permis de préciser le concept du jeu, les actions du personnage et le rôle des obstacles.

Ensuite, nous avons procédé à la conception de la structure du programme en identifiant les différentes classes nécessaires, telles que la classe du joueur, des obstacles, du menu et du jeu principal. Cette organisation nous a aidées à mieux répartir les responsabilités de chaque partie du programme.

Après cela, nous avons entamé la phase de développement, durant laquelle nous avons implémenté progressivement les différentes fonctionnalités du jeu, notamment le déplacement du personnage, la gestion des obstacles, les collisions, l'affichage graphique et les différents écrans du jeu.

Enfin, nous avons consacré une dernière étape aux tests et aux corrections afin d'améliorer le fonctionnement général de l'application, corriger les erreurs rencontrées et rendre le jeu plus fluide et plus cohérent.

1.5. Conclusion

En somme, ce chapitre nous a permis de présenter le contexte général de notre mini-projet, ainsi que les motivations et les objectifs visés. Nous avons également mis en évidence les principales difficultés rencontrées lors de la réalisation du jeu, notamment la gestion des déplacements du personnage, des collisions, de l’affichage graphique et de l’organisation du code.

Ce chapitre nous a ainsi permis de mieux comprendre les différents aspects liés au développement d’une application interactive et divertissante en C++. Dans le chapitre suivant, nous allons approfondir l’analyse du jeu ainsi que son fonctionnement.

Chapitre 2

Analyse du projet

2.1. Introduction

Dans ce chapitre, nous présentons une analyse détaillée de notre jeu de cachette en 2D. Cette étape est essentielle afin de mieux comprendre le fonctionnement global du jeu, les interactions entre ses différents éléments ainsi que les règles qui le régissent.

L'analyse du projet permet également d'identifier les principales fonctionnalités à implémenter, notamment le déplacement du personnage, la gestion des obstacles, la détection des collisions et l'affichage graphique. Elle constitue ainsi une base importante pour la conception et la réalisation de l'application.

2.2. Description du jeu

Le jeu de cachette en 2D est une application interactive dans laquelle le joueur contrôle un personnage évoluant dans un environnement dynamique. L'objectif principal du jeu est d'éviter les obstacles qui apparaissent progressivement tout au long du parcours.

Le joueur doit faire preuve de rapidité et de précision afin de réagir correctement face aux différents obstacles. Ces derniers peuvent apparaître soit au sol, soit en hauteur, ce qui oblige le joueur à adapter ses actions en fonction de la situation, notamment en sautant ou en s'accroupissant.

Le jeu propose également un écran de menu permettant à l'utilisateur de lancer une partie, de consulter la page "À propos" ou de quitter l'application. Avant de commencer la partie,

le joueur a la possibilité de choisir parmi plusieurs personnages, ce qui rend l'expérience plus personnalisée.

L'environnement du jeu est animé grâce à un fond défilant, donnant l'impression d'un mouvement continu. Un système de temps ou de score ou des tentatives restantes est également intégré afin de rendre le jeu plus motivant et d'évaluer la performance du joueur.

Ainsi, ce jeu offre une expérience simple, interactive et divertissante, permettant à l'utilisateur de passer le temps tout en relevant un défi progressif.

2.3. Fonctionnement du jeu

Le fonctionnement du jeu repose sur plusieurs étapes successives permettant d'assurer une interaction fluide entre l'utilisateur et l'application.

Au lancement du programme, l'utilisateur accède d'abord au menu principal, qui lui permet de choisir entre jouer, consulter la page "À propos" ou quitter le jeu. Après avoir sélectionné l'option "Jouer", il est dirigé vers un écran de choix de personnage.

Une fois le personnage sélectionné, la partie démarre. Le joueur contrôle son personnage à l'aide du clavier, en effectuant des actions telles que le saut ou l'accroupissement afin d'éviter les obstacles qui apparaissent progressivement.

Les obstacles se déplacent de manière continue vers le joueur, ce qui donne l'impression que le personnage avance dans l'environnement. Le joueur doit donc réagir rapidement pour éviter toute collision.

Pendant la partie, un système de temps ou de score est affiché afin de suivre la progression du joueur. et un système de tentative restante. Le jeu se termine dans deux cas : soit lorsque le joueur entre en collision avec un obstacle 3 fois, soit lorsque le temps imparti est écoulé. En cas de collision pour la 3ème fois, un écran de fin de partie est affiché. En revanche, si le joueur parvient à survivre jusqu'à la fin du temps sans entrer en collision avec un obstacle, un écran de victoire apparaît.

Ainsi, le fonctionnement du jeu repose sur une boucle continue combinant l'affichage, la gestion des événements et la mise à jour des éléments du jeu.

2.4. Règles du jeu

- Le jeu de cachette en 2D repose sur un ensemble de règles simples permettant de garantir une expérience claire et cohérente pour l'utilisateur.
- Le joueur doit éviter tous les obstacles présents sur son chemin. Ces obstacles peuvent apparaître soit au sol, soit en hauteur, ce qui l'oblige à adapter ses actions en fonction de la situation.
- Pour éviter les obstacles, le joueur utilise le clavier afin d'effectuer différentes actions. Il peut sauter pour éviter les obstacles situés au sol et s'accroupir pour éviter ceux placés en hauteur. Une mauvaise réaction peut entraîner une collision.
- Le joueur peut également mettre le jeu en pause à tout moment à l'aide de la touche prévue à cet effet. Pendant la pause, les déplacements, les obstacles et le temps sont momentanément arrêtés jusqu'à la reprise de la partie.
- Si une collision se produit entre le personnage et un obstacle, la partie se termine immédiatement et un écran de fin de partie est affiché. Le joueur doit alors recommencer une nouvelle partie s'il souhaite rejouer.
- Le jeu est également limité par un temps donné. Si le joueur parvient à survivre jusqu'à la fin du temps sans entrer en collision avec un obstacle, il remporte la partie et un écran de victoire est affiché.
- Ainsi, le respect de ces règles est essentiel pour progresser dans le jeu et obtenir un bon résultat..

2.5. Conclusion

Dans ce chapitre, nous avons présenté une analyse détaillée de notre jeu de cachette en 2D. Nous avons décrit le principe du jeu, son fonctionnement ainsi que les règles à respecter pour jouer. Cette analyse nous a permis de mieux comprendre les interactions entre les différents éléments du jeu, notamment le personnage, les obstacles et l'environnement graphique. Elle a également permis d'identifier les principales fonctionnalités nécessaires à la réalisation de l'application. Ainsi, cette étape constitue une base essentielle pour la phase

Chapitre 2. Analyse du projet

de conception et de développement du jeu. Dans le chapitre suivant, nous allons présenter les outils et technologies utilisés pour la réalisation de ce projet.

Chapitre 3

Outils et technologies utilisés

3.1. Introduction

Dans ce chapitre, nous présentons les principaux outils et technologies utilisés pour la réalisation de notre jeu de cache en 2D. Le choix de ces outils repose sur leur efficacité, leur simplicité d'utilisation et leur adaptation au développement d'applications graphiques interactives.

3.2. Le langage C++

Le langage C++ est un langage de programmation puissant et largement utilisé dans le développement d'applications et de jeux vidéo. Il offre de bonnes performances et permet une gestion fine des ressources, ce qui le rend particulièrement adapté aux applications nécessitant rapidité et efficacité.

De plus, le C++ prend en charge la programmation orientée objet, ce qui permet d'organiser le code en classes et de structurer le projet de manière claire et efficace. Dans notre projet, ce langage a été utilisé pour implémenter la logique du jeu, gérer les interactions entre les différents éléments et assurer le bon fonctionnement global de l'application.



FIG. 3.1 : Langage de programmation C++

3.3. La bibliothèque SFML

La bibliothèque **SFML** (*Simple and Fast Multimedia Library*) est une bibliothèque graphique conçue pour faciliter le développement d'applications multimédias et interactives. Elle fournit des outils simples pour gérer les fenêtres, afficher des images, traiter les événements du clavier et de la souris, ainsi que gérer le son.

Dans le cadre de notre projet, **SFML** a été utilisée pour créer la fenêtre du jeu, afficher les éléments graphiques, gérer les animations du personnage et des obstacles, ainsi que détecter les interactions avec l'utilisateur. Grâce à sa simplicité et à son efficacité, elle nous a permis de développer rapidement une application fonctionnelle et interactive.



FIG. 3.2 : Bibliothèque SFML

3.4. Environnement de développement : Visual Studio

Pour le développement de ce projet, nous avons utilisé l'environnement de développement intégré **Visual Studio**. Cet outil offre une interface moderne et intuitive permettant d'écrire, compiler et exécuter le code en C++ de manière efficace.

Visual Studio facilite également la gestion des fichiers du projet et permet de détecter et corriger rapidement les erreurs. Il offre en outre la possibilité d'intégrer des bibliothèques externes, telles que **SFML**, ce qui est essentiel pour le développement d'applications graphiques.

Grâce à cet environnement, nous avons pu organiser notre travail, tester les différentes fonc-

tionnalités du jeu et améliorer progressivement notre application.



FIG. 3.3 : Environnement de développement Visual Studio

3.5. Environnement de modélisation : StarUML

L'outil **StarUML** est un logiciel de modélisation utilisé pour concevoir des diagrammes UML. Il permet de représenter de manière claire et structurée les différents éléments d'un système, notamment à travers des diagrammes de classes, de séquence ou encore de cas d'utilisation.

Dans le cadre de notre projet, **StarUML** a été utilisé pour concevoir le diagramme de classes du jeu. Cet outil nous a permis de visualiser les différentes classes du système, leurs attributs, leurs méthodes ainsi que les relations entre elles. Grâce à cette modélisation, nous avons pu mieux organiser notre code et faciliter la compréhension globale du fonctionnement du jeu.



FIG. 3.4 : Environnement de modélisation StarUML

3.6. Conclusion

Dans ce chapitre, nous avons présenté les principaux outils et technologies utilisés pour la réalisation de notre projet. Le langage C++, la bibliothèque SFML et l'environnement Visual Studio ont joué un rôle essentiel dans le développement de cette application.

Dans le chapitre suivant, nous allons aborder la conception et la réalisation du jeu en détaillant la structure du programme et les différentes classes utilisées.

Chapitre 4

Conception et réalisation

4.1. Introduction

Dans ce chapitre, nous présentons la conception et la réalisation de notre jeu de cachette en 2D. Cette étape permet de décrire l'organisation du système, les différentes classes utilisées ainsi que leur rôle dans le fonctionnement global de l'application.

L'objectif de cette partie est de montrer comment les différents composants du projet interagissent entre eux pour assurer un fonctionnement fluide, structuré et interactif du jeu.

4.2. Architecture du projet

Le projet a été développé en adoptant une approche orientée objet. Cette méthode permet de structurer le programme en plusieurs classes distinctes, indépendantes mais complémentaires.

L'application repose principalement sur cinq classes : **Entite**, **Joueur**, **Obstacle**, **Menu** et **Jeu**. Chacune de ces classes possède un rôle bien défini et contribue au bon fonctionnement global du système.

Cette architecture facilite la maintenance du code, améliore sa lisibilité et simplifie l'ajout de nouvelles fonctionnalités.

4.3. Diagramme de classes

Le diagramme de classes présenté ci-dessous illustre la structure globale de l'application ainsi que les relations entre les différentes classes.

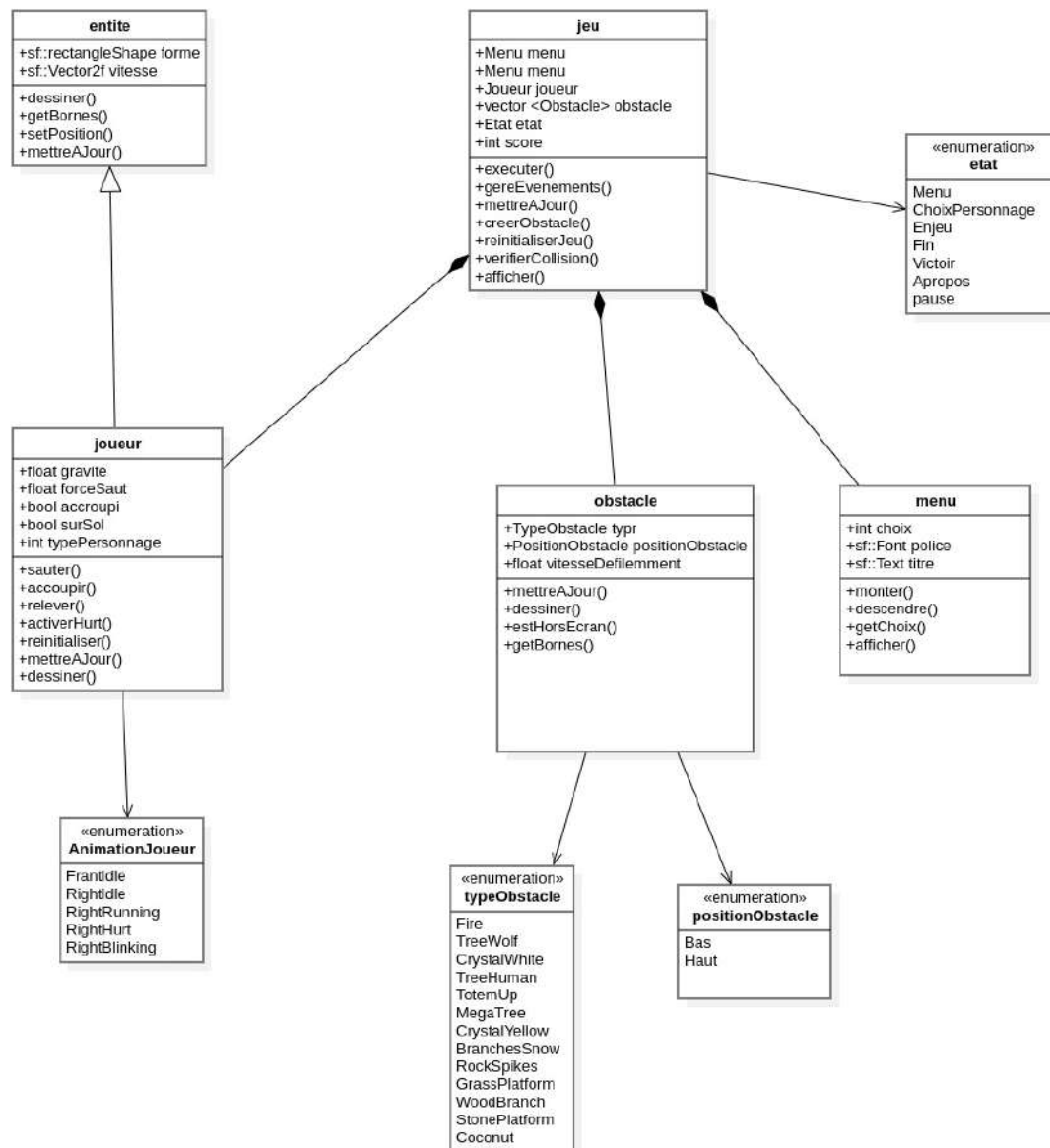


FIG. 4.1 : Diagramme de classes du jeu de cachette en 2D

4.4. Description du diagramme de classes

Le diagramme de classes met en évidence une organisation orientée objet claire et structurée.

La classe **Entite** représente une classe de base contenant les éléments communs aux objets du jeu, tels que la position, la vitesse et la forme graphique.

La classe **Joueur** hérite de la classe **Entite** et représente le personnage principal. Elle gère les déplacements, les animations ainsi que les interactions avec les obstacles.

La classe **Obstacle** représente les différents obstacles présents dans le jeu. Elle prend en charge leur type, leur position, leur déplacement ainsi que leur affichage.

La classe **Menu** permet de gérer l'interface utilisateur principale et les différentes options proposées au joueur.

La classe **Jeu** constitue le cœur du système. Elle gère la fenêtre, les événements, les états du jeu, le score, le temps ainsi que les interactions entre les différents éléments.

Le diagramme présente également plusieurs énumérations, telles que **Etat**, **TypeObstacle** et **PositionObstacle**, qui permettent de mieux organiser et simplifier la gestion des différents composants du jeu.

4.5. Fonctionnement du jeu

Le fonctionnement du jeu repose sur une boucle principale assurant la gestion des événements, la mise à jour des objets et l'affichage graphique.

Au démarrage, l'utilisateur accède au menu principal, où il peut choisir de lancer une partie, de consulter les informations du jeu ou de quitter l'application. Ensuite, il sélectionne un personnage avant de commencer la partie.

Pendant le jeu, le personnage évolue dans un environnement dynamique où apparaissent progressivement différents obstacles. Le joueur doit utiliser le clavier pour éviter ces obstacles, notamment en sautant ou en s'accroupissant selon la situation.

Le jeu se termine soit lorsqu'une collision est détectée entre le joueur et un obstacle, soit lorsque le temps imparti est écoulé. Dans ce dernier cas, le joueur remporte la partie.

4.6. Présentation des interfaces du jeu

Les figures suivantes illustrent les différentes étapes du jeu, depuis le menu principal jusqu'à la fin de la partie, en passant par la page « à propos », le choix du personnage et le déroulement du jeu.

4.6.1 Menu principal



FIG. 4.2 : Menu principal du jeu

Ce menu constitue le point d'entrée du jeu. Il permet à l'utilisateur de lancer une partie, consulter la page "À propos" ou quitter l'application.

4.6.2 Page À propos



FIG. 4.3 : Page « À propos » du jeu

Cette page présente des informations générales sur le jeu et permet à l'utilisateur de mieux comprendre son fonctionnement.

4.6.3 Choix du personnage



FIG. 4.4 : Choix du personnage

Cet écran permet au joueur de choisir un personnage avant de commencer la partie, ce qui rend l'expérience plus personnalisée.

4.6.4 Jeu en cours



FIG. 4.5 : Interface du jeu en cours

Cette interface montre le déroulement de la partie. Le joueur doit éviter les obstacles en contrôlant son personnage. Il a 3 tentatives si il entre en collision avec un obstacle mais il mort dans la 3eme fois.

4.6.5 Pause



FIG. 4.6 : Écran de pause du jeu

Cet écran apparaît lorsque le joueur met le jeu en pause. Il permet d'interrompre temporairement la partie.

4.6.6 Fin du jeu



FIG. 4.7 : Écran de fin du jeu (défaite)

Cet écran s'affiche lorsque le joueur entre en collision avec un obstacle.

4.6.7 Victoire



FIG. 4.8 : Écran de victoire

Cet écran apparaît lorsque le joueur réussit à survivre jusqu'à la fin du temps.

4.7. Conclusion

Dans ce chapitre, nous avons présenté la conception et la réalisation de notre jeu de cachette en 2D. Nous avons décrit l'architecture du projet, le diagramme de classes ainsi que le fonctionnement global du système.

L'utilisation de la programmation orientée objet nous a permis de structurer le projet de manière claire et efficace. Cette approche facilite la compréhension, la maintenance et l'évolution du code.

Conclusion et perspectives

Conclusion générale

Au terme de ce mini-projet, nous avons réussi à concevoir et à développer un **jeu de cache-cache en 2D** en utilisant le langage **C++** et la bibliothèque **SFML**. Ce travail nous a permis de mettre en pratique les connaissances acquises en programmation, notamment en programmation orientée objet, et de mieux comprendre le fonctionnement des applications graphiques interactives.

La réalisation de ce projet nous a confrontés à plusieurs défis, tels que la gestion des déplacements du personnage, la détection des collisions et l'organisation du code. Ces difficultés nous ont permis de développer notre capacité d'analyse et d'apprendre à résoudre progressivement différents problèmes techniques.

Ce mini-projet a également constitué une expérience enrichissante, car il nous a permis de renforcer nos compétences en développement logiciel, d'améliorer notre logique de programmation et de découvrir l'utilisation de la bibliothèque **SFML** dans la création d'applications 2D.

Perspectives

Dans le futur, plusieurs améliorations pourront être apportées à ce projet. Il serait possible d'ajouter de nouveaux niveaux de jeu, d'améliorer les graphismes et les animations, ou encore d'intégrer de nouvelles fonctionnalités telles que des effets sonores, un système de score plus complet ou une interface utilisateur plus avancée.

Ces perspectives permettent d'envisager une évolution du projet vers une application plus riche, plus complète et plus interactive.